

INSTITUTO SUPERIOR DE CIÊNCIAS E TECNOLOGIA DE MOÇAMBIQUE

Curso de Engenharia Informática

3.º Ano — Turma: A · 2026

Disciplina: Sistemas Distribuídos

RELATÓRIO DO TRABALHO PRÁTICO

AgroGest: Sistema de Gestão Agrícola em Java (Jakarta Servlets + JSP)

Nome do Estudante	Código do Estudante
Muhamad Shahin Riaz	20240568
Adayl Rafik Amad Seni	20240330
Kenny Matsule	20240908

Maputo, Outubro de 2026

Índice

1. Introdução

1.1 Objectivo geral · 1.2 Objectivos específicos · 1.3 Estrutura do relatório

2. Análise de requisitos

2.1 Perfis de utilizador · 2.2 Requisitos funcionais · 2.3 Requisitos não funcionais

3. Tecnologias utilizadas

4. Arquitectura do sistema

4.1 Padrão MVC · 4.2 Visão geral das camadas · 4.3 Fluxo de um pedido

5. Estrutura do projecto

5.1 Árvore de pastas · 5.2 Pacotes Java

6. Modelo de dados

6.1 Entidades · 6.2 Relações entre entidades · 6.3 Armazenamento em memória

7. Controladores e rotas

8. Regras de negócio

8.1 Venda atómica e emissão de factura

9. Segurança, concorrência e persistência

9.1 Autenticação e controlo de acesso · 9.2 Concorrência · 9.3 Persistência tolerante a falhas

10. Instalação, configuração de portas e execução

10.1 Requisitos · 10.2 Compilar e publicar · 10.3 Portas e systemd em Linux · 10.4 Contas de teste

11. Conclusão

11.1 Limitações · 11.2 Trabalho futuro

Referências bibliográficas

1. Introdução

O presente relatório descreve o desenvolvimento do **AgroGest**, uma aplicação web de gestão agrícola desenvolvida no âmbito da disciplina de Sistemas Distribuídos. O sistema responde às necessidades de um agricultor da província de Nampula, que precisa de gerir fornecedores de sementes, fertilizantes, maquinaria e ferramentas, controlar os cultivos e colheitas de cada campanha agrícola, vender produtos em stock a clientes, emitir facturas detalhadas com IVA (16%) e acompanhar relatórios de clientes e vendas.

A aplicação foi implementada em Java 17, com Jakarta Servlets 6.0 como controladores e JSP 3.1 + JSTL 3.0 como vistas, sendo publicada no servidor Apache Tomcat 10.1. Por restrição explícita do enunciado, não se utiliza base de dados relacional: os dados vivem em listas sincronizadas em memória (`Collections.synchronizedList`) e são persistidos em ficheiro (`agrogest_state.ser`) através da serialização nativa de Java.

1.1 Objectivo geral

Desenvolver uma aplicação web em Java, segundo o padrão arquitectural MVC, que permita gerir a produção agrícola por campanhas e a comercialização de insumos, com controlo de acesso por perfil, sincronização concorrente e persistência tolerante a falhas em ficheiro.

1.2 Objectivos específicos

- Estruturar a aplicação segundo o padrão Model-View-Controller (MVC);
- Implementar Servlets como controladores, JSP como vistas e POJOs serializáveis como modelo de domínio;
- Gerir sessões HTTP para autenticação segura e manutenção do carrinho de compras;
- Controlar o acesso às páginas por perfil através de um filtro (`AuthFilter`) com código HTTP 403;
- Trabalhar com listas sincronizadas em memória (`ArrayList`, `Map`, `Collections.synchronizedList`) e geradores atómicos (`AtomicInteger`);
- Garantir transações lógicas na venda com desconto de stock atómico (tudo ou nada);
- Persistir o estado do sistema em ficheiro com escrita atómica (`.tmp`) e cópia de segurança (`.bak`);
- Empacotar a aplicação num ficheiro WAR, configurar portas no Apache Tomcat e gerir o serviço via `systemd`.

1.3 Estrutura do relatório

O capítulo 2 apresenta os requisitos funcionais e não funcionais, e o capítulo 3 lista as tecnologias utilizadas. Os capítulos 4 e 5 descrevem a arquitectura MVC e a estrutura de pacotes do projecto. O capítulo 6 detalha o modelo de domínio, o capítulo 7 os controladores e rotas, e o capítulo 8 as regras de negócio. O capítulo 9 aborda a segurança, concorrência e persistência em ficheiro, o capítulo 10 explica a instalação, configuração de portas e execução com `systemd`, e o capítulo 11 apresenta a conclusão.

2. Análise de requisitos

2.1 Perfis de utilizador

Perfil	Descrição
Administrador	Conta fixa criada automaticamente no arranque do sistema (`admin@agro.mz` / `admin234`). Gere todo o catálogo de produtos, fornecedores, campanhas, cultivos, colheitas, clientes, vendas, relatórios e persistência.
Cliente	Auto-regista-se (ou é registado pelo administrador). Consulta o catálogo com filtros e pesquisa, adiciona produtos ao carrinho em sessão, efectua compras e consulta ou imprime as suas facturas.

Tabela 1 – Perfis de utilizador

2.2 Requisitos funcionais

Código	Requisito	Perfil
RF01	Autenticar com email e palavra-passe (com renovação do identificador de sessão)	Ambos
RF02	Auto-registar conta de cliente (nome, email único, palavra-passe mín. 4 caracteres, telefone, endereço)	Cliente
RF03	Criar, listar, ordenar e remover fornecedores (`SEMENTES` , `MAQUINARIA` , `FERTILIZANTES`)	Admin
RF04	Criar, listar, filtrar, ordenar, repor stock e remover produtos (`SEMENTE` , `FERTILIZANTE` , `MAQUINARIA` , `FERRAMENTA`)	Admin
RF05	Criar, fechar, reabrir e remover campanhas agrícolas (impedindo remoção se tiver cultivos)	Admin
RF06	Registar e remover cultivos por campanha (impedindo remoção se tiver colheitas)	Admin
RF07	Registar colheitas por cultivo (validando data posterior ao plantio) com cálculo automático do valor total	Admin
RF08	Listar produtos, fornecedores, clientes, produtos em armazém (`stock > 0`) e compras detalhadas com ordenação	Admin
RF09	Registar clientes e vendas presenciais ao balcão	Admin
RF10	Emitir relatórios de clientes: ordenados por nome, por volume de compras, mais assíduos e clientes sem compras (extra)	Admin
RF11	Gravar e eliminar manualmente o ficheiro de estado serializado, além da gravação automática	Admin
RF12	Consultar o catálogo de produtos disponíveis, filtrar por categoria, pesquisar e ordenar	Cliente
RF13	Gerir o carrinho de compras em sessão (adicionar, actualizar quantidades, remover itens, esvaziar)	Cliente
RF14	Confirmar compra com verificação e desconto atómico de stock e emissão de factura sequencial (`FT-AAAA-N`) com IVA (16%)	Cliente
RF15	Consultar o histórico de compras anteriores e visualizar/imprimir as respectivas facturas	Cliente

Tabela 2 – Requisitos funcionais

2.3 Requisitos não funcionais

Código	Requisito
RNF01	Interface web em português, responsiva e com folhas de estilo CSS próprias
RNF02	Arquitectura MVC clara com código organizado em pacotes por responsabilidade (`model` , `repo` , `service` , `servlet` , `filter` , `util` , `listener`)
RNF03	Operações de escrita concorrentes protegidas por blocos `synchronized` e listas sincronizadas
RNF04	Nenhuma palavra-passe em texto puro: armazenamento através de hash criptográfico SHA-256 e protecção XSS com `<c:out>`
RNF05	Persistência tolerante a falhas com escrita atómica via ficheiro temporário (`.tmp`) e cópia de segurança (`.bak`)
RNF06	Compatibilidade com Apache Tomcat 10.1 (Jakarta EE 10) e Java 17+
RNF07	Armazenamento exclusivamente em memória e ficheiro serializado (sem SGBD relacional)

Tabela 3 – Requisitos não funcionais

3. Tecnologias utilizadas

Camada / Função	Tecnologia	Versão
Linguagem de programação	Java SE (JDK)	17 (compatível com 17+)
Servidor de aplicações	Apache Tomcat	10.1
Controladores	Jakarta Servlets (`@WebServlet`)	6.0
Filtros e Listeners	`jakarta.servlet.Filter` (`@WebFilter`) e `ServletContextListener` (`@WebListener`)	6.0
Vista	JSP + JSTL + Expression Language (EL) + HTML5 + CSS3 + JS	JSP 3.1 / JSTL 3.0
Modelos e Persistência em Memória	POJOs `Serializable` , `ArrayList` , `Map` , `Collections.synchronizedList` , `AtomicInteger`	Java SE 17
Persistência em Disco	`ObjectOutputStream` / `ObjectInputStream` (`agrogest_state.ser`)	Java SE 17
Construção e Empacotamento	Apache Maven (empacotamento `war`)	3.9

Tabela 4 – Tecnologias utilizadas

4. Arquitectura do sistema

4.1 Padrão MVC

A aplicação segue rigorosamente o padrão **Model-View-Controller (MVC)**, separando os dados e regras de negócio (Model), a interface gráfica (View) e o controlo de pedidos HTTP (Controller):

Camada	Responsabilidade	Implementação no AgroGest
Model	Representa as entidades de negócio, o repositório em memória e as regras transaccionais	Pacotes `model`, `repo` e `service`
View	Apresenta os dados formatados ao utilizador sem conter lógica de negócio	Ficheiros JSP em `/WEB-INF/views/` + CSS/JS
Controller	Recebe pedidos HTTP (GET/POST), valida parâmetros, invoca os serviços e encaminha para a vista	Servlets nos pacotes `servlet` e `servlet.admin`

Tabela 5 – Camadas MVC

Todas as vistas internas residem dentro de `/WEB-INF/views/`, impedindo o acesso directo pelo navegador e garantindo que qualquer pedido passa primeiro pelo filtro de segurança e pelo respectivo Servlet controlador.

4.2 Visão geral das camadas

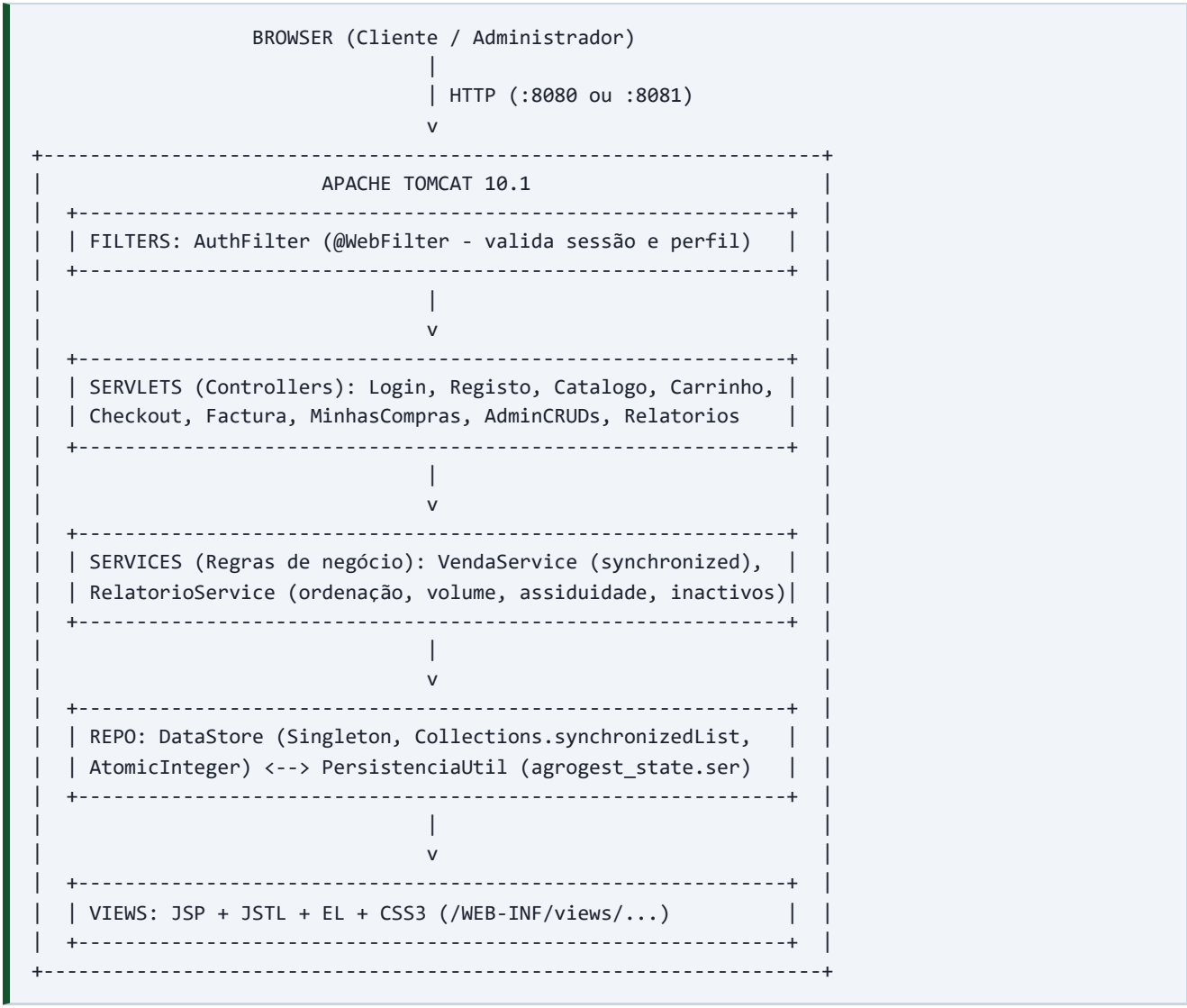


Figura 1 – Arquitectura em camadas do sistema AgroGest

4.3 Fluxo de um pedido (Checkout de Compra)

1. O cliente autenticado clica em confirmar compra, enviando `POST /compra`.
2. O `AuthFilter` intercepta o pedido e confirma que existe sessão HTTP válida.
3. O `CheckoutServlet` verifica que o utilizador na sessão é uma instância de `Cliente`, lê o carrinho da sessão e invoca `VendaService.efetuarVenda(cliente, itens)`.
4. O `VendaService` sincroniza no `DataStore` (`synchronized (store)`), valida se todos os produtos existem e têm stock suficiente, desconta o stock atomicamente, gera a factura com número sequencial `FT-2026-N`, calcula subtotal, IVA (16%) e total, adiciona a venda ao `DataStore` e grava o estado em disco via `PersistenciaUtil.gravar()`.
5. O `CheckoutServlet` limpa o carrinho da sessão e redirecciona (Post-Redirect-Get) para `GET /factura?id=N`.
6. O `FacturaServlet` valida que a factura pertence ao cliente autenticado (RN03) e encaminha para `/WEB-INF/views/cliente/factura.jsp`.

5. Estrutura do projecto

5.1 Árvore de pastas

```
agrogest/
├── pom.xml                                  Configuração Maven (Java 17, Jakarta EE 10, WAR)
├── README.md                                Instruções de instalação, deploy, portas e arquitectura
├── run.bat / stop.bat                       Scripts de compilação e execução rápida (Windows)
├── docs/
│   ├── ENUNCIADO_TRABALHO_PRATICO.md        Enunciado oficial do trabalho prático
│   ├── MANUAL_DE_UTILIZACAO.pdf             Manual de utilização passo a passo (Admin e Cliente)
│   ├── RELATORIO_TRABALHO_PRATICO.pdf       Relatório técnico do projecto
│   └── tomcat.service                       Unidade systemd para gestão do Tomcat em Linux
└── src/main/
    ├── java/mz/ac/istcem/agrogest/
    │   ├── model/                           POJOs Serializable (Utilizador, Produto, Venda, etc.)
    │   ├── repo/                            DataStore (Singleton) e EstadoSistema (snapshot)
    │   ├── service/                         VendaService e RelatorioService
    │   ├── servlet/                         Controladores de autenticação e módulo Cliente
    │   │   └── admin/                       Controladores do módulo Administrador
    │   ├── filter/                          AuthFilter (controlo de sessão e perfil HTTP 403)
    │   ├── listener/                        AppListener (arranque, seed inicial e encerramento)
    │   └── util/                            PasswordUtil, PersistenciaUtil e GraficoUtil
    └── webapp/
        ├── index.jsp                        Redirecciona para /login.jsp
        ├── login.jsp / registo.jsp          Páginas públicas de autenticação e auto-registo
        ├── erro403.jsp                      Página de acesso negado (HTTP 403 Forbidden)
        ├── css/style.css                    Folha de estilos responsiva própria
        ├── js/graficos.js                   Renderização de gráficos SVG no dashboard
        └── WEB-INF/
            ├── web.xml                      Configuração UTF-8, timeout de sessão (30 min) e erro 4
            └── views/
                ├── _partials/               Cabeçalho, barra lateral admin e topo cliente
                ├── admin/                   Dashboard, produtos, fornecedores, campanhas, etc.
                └── cliente/                 Catálogo, carrinho, factura e histórico de compras
```

Figura 2 – Árvore de pastas do projecto

5.2 Pacotes Java

Pacote	Classes	Responsabilidade
`model`	`Utilizador`, `Administrador`, `Cliente`, `Fornecedor`, `Produto`, `Campanha`, `Cultivo`, `Colheita`, `Venda`, `VendaItem`	Entidades de domínio (POJOs que implementam `Serializable`)
`repo`	`DataStore`, `EstadoSistema`	Repositório em memória (`synchronizedList` + `AtomicInteger`) e snapshot de serialização
`service`	`VendaService`, `RelatorioService`	Transação atómica de venda com IVA 16% e geração dos 4 relatórios de clientes
`servlet`	`LoginServlet`, `LogoutServlet`, `RegistoServlet`, `CatalogoServlet`, `CarrinhoServlet`, `CheckoutServlet`, `FacturaServlet`, `MinhasComprasServlet`	Controladores de autenticação e operações do cliente
`servlet.admin`	`AdminDashboardServlet`, `ProdutoServlet`, `FornecedorServlet`, `CampanhaServlet`, `CultivoServlet`, `ColheitaServlet`, `ClienteAdminServlet`, `VendaAdminServlet`, `RelatorioServlet`, `PersistenciaServlet`	Controladores exclusivos do perfil Administrador (`/admin/*`)
`filter`	`AuthFilter`	Intercepta rotas protegidas, exige sessão e devolve HTTP 403 a não-administradores em `/admin/*`
`listener`	`AppListener`	Carrega `agrogest_state.ser` no arranque, popula dados iniciais se vazio e grava no encerramento
`util`	`PasswordUtil`, `PersistenciaUtil`, `GraficoUtil`	Hash SHA-256, escrita atómica em ficheiro com backup `.bak` e geração de séries JSON para gráficos

Tabela 6 – Pacotes e classes Java (37 ficheiros Java e 21 vistas JSP)

6. Modelo de dados

6.1 Entidades

Todas as entidades do pacote `mz.ac.istcem.agrogest.model` seguem a convenção JavaBean (POJO) e implementam `java.io.Serializable` com `serialVersionUID` explícito:

Classe	Atributos principais
`Utilizador` (abstracta)	`id`, `nome`, `email`, `passwordHash`, `perfil` (`ADMIN` / `CLIENTE`), `ativo`
`Administrador`	Herda de `Utilizador` (perfil fixo `ADMIN`)
`Cliente`	Herda de `Utilizador` + `telefone`, `endereco`
`Fornecedor`	`id`, `nome`, `tipo` (`SEMENTES`, `MAQUINARIA`, `FERTILIZANTES`), `contacto`, `provincia`
`Produto`	`id`, `nome`, `categoria` (`SEMENTE`, `FERTILIZANTE`, `MAQUINARIA`, `FERRAMENTA`), `fornecedorId`, `fornecedorNome`, `preco`, `quantidade`, `unidade`, `descricao`, `fotoUrl`, `validade`
`Campanha`	`id`, `nome`, `inicio` (`LocalDate`), `fim` (`LocalDate`), `aberta` (`boolean`)
`Cultivo`	`id`, `campanhaId`, `cultura`, `variedade`, `hectares`, `dataPlantio` (`LocalDate`)
`Colheita`	`id`, `cultivoId`, `data` (`LocalDate`), `quantidadeKg`, `precoPorKg`, `valorTotal` (calculado: `quantidadeKg * precoPorKg`)
`Venda`	`id`, `numeroFactura` (`FT-AAAA-N`), `clienteId`, `clienteNome`, `clienteEndereco`, `clienteTelefone`, `data` (`LocalDateTime`), `itens` (`List<VendaItem>`), `subtotal`, `iva` (16%), `total`
`VendaItem`	`produtoId`, `produtoNome`, `quantidade`, `precoUnitario`, `subtotal`

Tabela 7 – Entidades do domínio e respectivos atributos

6.2 Relações entre entidades

```
Utilizador
  ├── Administrador
  └── Cliente (1) ──────── (*) Venda (1) ──────── (*) VendaItem (*) ─── (1) Produto
                                                                              (*)
Fornecedor (1) ──────────────────────────────────────────────────────────────┘

Campanha (1) ──────── (*) Cultivo (1) ──────── (*) Colheita
```

Figura 3 – Diagrama de relações entre entidades

6.3 Armazenamento em memória

A classe `DataStore` implementa o padrão *Singleton* (`DataStore.getInstance()`). Cada colecção de entidades é mantida numa lista envolvida por `Collections.synchronizedList(new ArrayList<>())` e possui o seu próprio gerador sequencial `AtomicInteger` (`utilizadorSeq`, `fornecedorSeq`, `produtoSeq`, `campanhaSeq`, `cultivoSeq`, `colheitaSeq`, `vendaSeq` e `numeroFactura`), cumprindo a regra de negócio RN06.

7. Controladores e rotas

Servlet	Rota (`@WebServlet`)	Métodos	Operações suportadas
`LoginServlet`	`/login`	GET, POST	Valida credenciais (SHA-256), invoca `req.changeSessionId()` e inicia sessão
`LogoutServlet`	`/logout`	GET	Invalida a sessão HTTP actual e redirecciona para `/login.jsp`
`RegistoServlet`	`/registo`	GET, POST	Auto-registo de cliente (email único, senha mín. 4 caracteres) e gravação imediata
`CatalogoServlet`	`/catalogo`	GET	Lista produtos com filtro por categoria, pesquisa textual e ordenação
`CarrinhoServlet`	`/carrinho`, `/carrinho/*`	GET, POST	Acções: `adicionar`, `atualizar`, `remover`, `limpar` (em sessão)
`CheckoutServlet`	`/compra`	POST	Verifica perfil Cliente, executa `VendaService.efetuarVenda()` e redirecciona para factura
`FacturaServlet`	`/factura`	GET	Exibe factura detalhada (restrita ao cliente dono da compra ou ao administrador)
`MinhasComprasServlet`	`/minhas-compras`	GET	Lista histórico de compras e facturas do cliente autenticado
`AdminDashboardServlet`	`/admin/dashboard`	GET	KPIs gerais, alertas de stock e gráficos analíticos
`ProdutoServlet`	`/admin/produtos`	GET, POST	Criar produto, repor stock, remover, filtrar por armazém (`stock > 0`) e ordenar
`FornecedorServlet`	`/admin/fornecedores`	GET, POST	Criar, listar com ordenação e remover fornecedores
`CampanhaServlet`	`/admin/campanhas`	GET, POST	Criar (validando datas), fechar, reabrir e remover campanhas sem cultivos
`CultivoServlet`	`/admin/cultivos`	GET, POST	Criar e remover cultivos sem colheitas associadas
`ColheitaServlet`	`/admin/colheitas`	GET, POST	Registar colheita (validando data ≥ data de plantio) e calcular valor total
`ClienteAdminServlet`	`/admin/clientes`	GET, POST	Listar, ordenar, criar e remover clientes sem compras associadas
`VendaAdminServlet`	`/admin/vendas`	GET, POST	Listar compras detalhadas com ordenação e registar venda ao balcão
`RelatorioServlet`	`/admin/relatorios`	GET	Relatórios de clientes (por nome, por volume, mais assíduos e sem compras)

Servlet	Rota (`@WebServlet`)	Métodos	Operações suportadas
`PersistenciaServlet`	`/admin/persistencia`	POST	Acções manuais: `gravar` e `eliminar` ficheiro de estado

Tabela 8 – Mapeamento de Servlets, rotas e operações

8. Regras de negócio

Código	Regra de Negócio	Implementação
RN01	Não é possível vender mais do que o stock disponível	Validação prévia em `CarrinhoServlet` e verificação estrita em `VendaService.efetuarVenda()`
RN02	Uma venda desconta o stock de todos os itens atomicamente (tudo ou nada)	Bloco `synchronized (store)` em duas fases (1.ª validação completa; 2.ª desconto de stock) em `VendaService`
RN03	Um cliente só vê as suas próprias facturas; o administrador vê todas	Verificação `!"ADMIN".equals(u.getPerfil()) && venda.getClienteId() != u.getId()` no `FacturaServlet`
RN04	Rotas `/admin/*` são exclusivas do administrador (HTTP 403 caso contrário)	`AuthFilter` define `res.setStatus(HttpServletResponse.SC_FORBIDDEN)` e encaminha para `/erro403.jsp`
RN05	Rotas `/catalogo`, `/carrinho/*`, `/compra`, `/factura`, `/minhas-compras` exigem sessão	`AuthFilter` verifica sessão activa e redirecciona para `/login.jsp` se não autenticado
RN06	IDs de entidades são sequenciais e únicos, gerados por `AtomicInteger`	Contadores `AtomicInteger` dedicados no `DataStore`
Integridade	Proibido remover campanhas com cultivos, cultivos com colheitas ou clientes com vendas	Validações de integridade referencial em `CampanhaServlet`, `CultivoServlet` e `ClienteAdminServlet`
Datas e Valores	Fim da campanha ≥ início; data da colheita ≥ data de plantio; preços e quantidades positivos	Validações com tratamento de excepções (`NumberFormatException`, `DateTimeParseException`) nos Servlets

Tabela 9 – Regras de negócio e respectiva implementação

8.1 Venda atómica e emissão de factura

O método `VendaService.efetuarVenda()` sincroniza sobre a instância única do `DataStore` e executa o algoritmo em duas passagens: primeiro garante que todos os produtos existem e possuem stock suficiente; só se nenhum falhar é que desconta as quantidades, emite a factura no formato `FT-AAAA-N` (ex.: `FT-2026-1`), calcula o IVA de 16% e persiste imediatamente o estado:

```
public Venda efetuarVenda(Cliente cliente, List<VendaItem> itens) {
    if (itens == null || itens.isEmpty()) {
        throw new IllegalArgumentException("O carrinho está vazio.");
    }
    DataStore store = DataStore.getInstance();
    synchronized (store) {
        // 1.ª passagem: validar existência e stock de TODOS os itens
        for (VendaItem item : itens) {
            Produto p = store.findProduto(item.getProdutoId());
            if (p == null) {
                throw new IllegalStateException("Produto inexistente: " + item.getProdutoNome());
            }
            if (p.getQuantidade() < item.getQuantidade()) {
                throw new IllegalStateException("Stock insuficiente para '" + p.getNome()
                        + "' (disponível: " + p.getQuantidade() + ", pedido: " + item.getQuantidade(
            }
        }
        // 2.ª passagem: descontar stock atomicamente
        for (VendaItem item : itens) {
            Produto p = store.findProduto(item.getProdutoId());
            p.setQuantidade(p.getQuantidade() - item.getQuantidade());
        }
        LocalDateTime agora = LocalDateTime.now();
        String numFactura = "FT-" + agora.getYear() + "-" + store.numeroFactura.getAndIncrement();
        Venda venda = new Venda(store.vendaSeq.getAndIncrement(), numFactura,
                cliente.getId(), cliente.getNome(), cliente.getEndereco(), cliente.getTelefone(),
                agora, new ArrayList<>(itens));
        venda.calcularTotais(); // subtotal + IVA (16%) + total
        store.vendas.add(venda);
        PersistenciaUtil.gravar();
        return venda;
    }
}
```

Figura 4 – Implementação da venda transaccional atómica em `VendaService.java`

9. Segurança, concorrência e persistência

9.1 Autenticação e controlo de acesso

- **Hash de palavras-passe:** Todas as senhas são processadas com SHA-256 (`PasswordUtil.hash()`) antes de serem guardadas ou comparadas.
- **Prevenção de Session Fixation:** No momento do login com sucesso, `LoginServlet` invoca `req.changeSessionId()` antes de associar o utilizador à sessão.
- **Controlo de acesso e HTTP 403:** O `AuthFilter` protege todas as rotas de cliente e de administração. Se um cliente tentar aceder a `/admin/*` ou a `/factura` de outro cliente, o servidor responde com

código HTTP `403 Forbidden` e apresenta `erro403.jsp`.

- **Prevenção de XSS:** Todos os dados dinâmicos e parâmetros reflectidos nas JSP são escapados com `<c:out>`.

9.2 Concorrência

Como o Apache Tomcat processa múltiplos pedidos HTTP em paralelo através de um pool de threads, o estado partilhado no `DataStore` está protegido a três níveis: (1) listas encapsuladas com `Collections.synchronizedList`; (2) sincronização explícita sobre o repositório (`synchronized (store)`) na venda atómica e no snapshot de estado; e (3) contadores `AtomicInteger` livres de condições de corrida.

9.3 Persistência tolerante a falhas

A classe `PersistenciaUtil` gere o ficheiro `agrogest_state.ser` (localizado em `${catalina.base}/agrogest_state.ser`):

- **Gravação atómica e contínua:** Sempre que ocorre uma alteração (venda, criação, reposição ou remoção), no botão manual ou no encerramento do Tomcat (`AppListener.contextDestroyed`), o estado é primeiro serializado para `agrogest_state.ser.tmp`; o ficheiro anterior é copiado para `agrogest_state.ser.bak` e só depois o `.tmp` substitui atomicamente o ficheiro principal (`ATOMIC_MOVE`).
- **Recuperação automática de falhas:** No arranque (`AppListener.contextInitialized`), se o ficheiro principal estiver corrompido, `PersistenciaUtil.carregar()` isola-o como `.corrompido` e restaura automaticamente a cópia íntegra a partir de `agrogest_state.ser.bak`.

10. Instalação, configuração de portas e execução

10.1 Requisitos

Java JDK 17 ou superior, Apache Maven 3.9+ e Apache Tomcat 10.1.

10.2 Compilar e publicar

```
mvn clean package
cp target/agrogest.war /opt/tomcat/webapps/
```

Em Windows, basta executar `run.bat` na raiz do projecto para compilar, publicar no Tomcat e iniciar o servidor automaticamente.

10.3 Configuração de portas e serviço systemd em Linux

Para alterar a porta HTTP do Tomcat (por exemplo, de `8080` para `8081`, conforme o diagrama da arquitectura no enunciado), edita-se o ficheiro `conf/server.xml` do Tomcat:

```
<Connector port="8081" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443"
           URIEncoding="UTF-8" />
```

Para gerir o Tomcat como serviço `systemd` em Linux (usando o ficheiro incluído em `docs/tomcat.service`):

```
sudo cp docs/tomcat.service /etc/systemd/system/tomcat.service
sudo systemctl daemon-reload
sudo systemctl enable tomcat
sudo systemctl start tomcat
sudo systemctl status tomcat
```

10.4 Contas de teste

Perfil	Email	Palavra-passe	Observações
Administrador	`admin@agro.mz`	`admin234`	Acesso total ao módulo `/admin/*`
Cliente (Enunciado)	`joao@teste.mz`	`1234`	João Silva — Av. Eduardo Mondlane, Nampula
Cliente	`cliente@agro.mz`	`cliente123`	Amina Momade — Cidade de Nampula
Cliente (Sem compras)	`helena@monapo.mz`	`cliente123`	Para testar o relatório extra de clientes sem compras

Tabela 10 – Principais credenciais de teste criadas no arranque

11. Conclusão

O sistema **AgroGest** cumpre integralmente todos os requisitos funcionais, não funcionais e regras de negócio (RN01 a RN06) definidos no enunciado. A arquitectura MVC com Jakarta Servlets, JSP/JSTL, filtro de segurança e camada de serviços sincronizada garante uma clara separação de responsabilidades, protecção contra acessos não autorizados e consistência transaccional nas vendas concorrentes.

11.1 Limitações

- Por restrição académica do enunciado, todos os dados residem em memória RAM (com persistência em ficheiro único), o que limita a escalabilidade horizontal em múltiplos nós;
- O hash SHA-256 protege contra leitura de senhas em texto simples, embora em sistemas bancários ou críticos de produção seja recomendável um algoritmo adaptativo com salt (como BCrypt ou Argon2).

11.2 Trabalho futuro

- Migrar a camada `DataStore` para uma base de dados relacional (PostgreSQL) via JPA/Hibernate;
- Integrar pagamentos móveis (M-Pesa / e-Mola) no fluxo de checkout do cliente;
- Adicionar autenticação com BCrypt e tokens anti-CSRF em todos os formulários de administração.

Referências bibliográficas

1. Eclipse Foundation. *Jakarta Servlet Specification, Version 6.0*. Disponível em: https://jakarta.ee/specifications/servlet/6.0/
2. Eclipse Foundation. *Jakarta Server Pages Specification, Version 3.1*. Disponível em: https://jakarta.ee/specifications/pages/3.1/

3. The Apache Software Foundation. *Apache Tomcat 10.1 Documentation*. Disponível em: https://tomcat.apache.org/tomcat-10.1-doc/
4. Oracle. *Java SE 17 API Documentation* (`java.io.Serializable` , `java.util.Collections` , `java.util.concurrent.atomic.AtomicInteger`).
5. Coulouris, G., Dollimore, J., Kindberg, T. & Blair, G. (2012). *Distributed Systems: Concepts and Design* (5.ª ed.). Addison-Wesley.
6. ISCTEM (2026). *Documento de Enunciado para Trabalho Prático — AgroGest: Sistema de Gestão Agrícola*. Disciplina de Sistemas Distribuídos.